VAULTLY MASTER ENGINEERING SERIES • CHAPTER 1

Vaultly: Complete Project Understanding

Software Architecture Blueprint & Foundation Lesson for Full-Stack Mastery

APPLICATION	ARCHITECTURE	ROLE	STORAGE LAYER
Vaultly Expense Tracker	Vanilla SPA + Serverless AI	Senior Architect & Mentor	Client-Side LocalStorage

1 What Vaultly Actually Is (Audited from Code)

Vaultly is a privacy-first, client-side personal finance and financial intelligence **Single Page Application (SPA)** paired with an on-demand serverless conversational AI assistant.

> **The Problem It Solves:** Most budgeting and finance apps require users to upload bank logins, credit card statements, and daily cash transactions to unknown third-party cloud databases. Vaultly eliminates that risk entirely: **100% of financial transactions, budgets, recurring obligations, and savings goals remain on the user's local machine in browser** `localStorage`.

Core Capabilities Discovered in Code:

- **Personal Finance Management:** Multi-currency transaction logging, categorized budgets, multi-tier savings goals, and recurring payment tracking.
- **Financial Intelligence Engines:**
  - **4-Pillar Financial Health Score:** Evaluates Savings Rate (30%), Budget Adherence (30%), Runway Days (25%), and Volatility (15%).
  - **What-If Digital Twin:** Simulates cashflow scenarios before users make discretionary purchases.
  - **Anomaly Detection Engine:** Flags statistical expense spikes compared against historical category averages.
  - **Safe-to-Spend Runway:** Computes true disposable capital after deducting fixed commitments.
- **Conversational AI Assistant:** Seamlessly queries real financial context when finance topics are discussed, while remaining a versatile assistant for coding, writing, learning, and general problem-solving.

2 High-Level Architecture Diagram

Vaultly separates the privacy-safe user interface from cloud AI using an intelligent dual-tier architecture:

TIER 1: Client-Side Browser (100% Private)

• **DOM Viewport (index.html):** 7 view panels, modal dialogs, and SVG icons.
• **Presentation Layer (style.css):** Obsidian Glass monochrome design system.
• **Application Controller (script.js):** Central reactive state object.
• **Persistence:** `localStorage` (Key: `vaultly_state_v2`).
• **Analytics:** HTML5 Canvas via Chart.js v4.4.1.

TIER 2: Cloud Serverless (api/chat.js)

• **Vercel Serverless Function:** Secure Node.js proxy environment.
• **In-Memory Rate Limiter:** Prevents API flooding and token exhaustion.
• **Google Gemini API:** Primary fast conversational LLM.
• **OpenAI API:** Secondary fallback LLM.
• **Deterministic Offline Engine:** Instant rule-based answers when offline.

Data Flow: User Click ➔ script.js Controller ➔ LocalStorage ➔ Math Engines ➔ DOM Update ➔ Optional /api/chat Proxy

3 The Actual Technology Stack

LAYER	TECHNOLOGY DISCOVERED	ARCHITECTURAL RATIONALE
Frontend Core	Semantic HTML5, CSS3, Modern Vanilla JS (ES2022+)	Zero framework overhead (no React/Next.js client bloat). Instant 0ms hydration and minimal CPU footprint.
Build Tooling	Zero-Build Architecture	No root `package.json` or compiler needed. Runs natively in any standard browser immediately.
Design System	Obsidian Glass Monochrome	CSS custom properties, glassmorphism backdrops, smooth cubic-bezier transitions, and high-DPI responsive design.
Iconography	Lucide Icons CDN (v0.344.0)	Lightweight SVG vector icons replaced dynamically via client-side DOM scan (`lucide.createIcons()`).
Visual Charts	Chart.js CDN (v4.4.1)	Hardware-accelerated HTML5 `<canvas>` rendering for doughnut distributions and bar charts.
Storage	Browser `localStorage`	Zero external database dependency. Complete data ownership and offline accessibility for users.
Backend Gateway	Vercel Serverless (Node.js)	Houses `api/chat.js` to keep secret AI API keys safe on the server side instead of exposing them in client JS.
AI Models	Google Gemini & OpenAI	Gemini 1.5 Flash / 2.0 Flash with secondary OpenAI `gpt-4o-mini` support and deterministic offline fallback.
Quality Harness	Node.js Testing Scripts	19 unit tests in `test-runner.js` ; 14-device viewport responsive auditor in `verify-responsive.mjs` .

4 Simplified Project Folder Tree

```
vaultly-expense-tracker/
├── index.html                  # Single-Page Application DOM structure & 7 view panels
├── style.css                   # Obsidian Glass styling, responsive layout & animations
├── script.js                   # Reactive state manager, math engines & UI controller
├── vercel.json                 # Routing configuration & HTTP security headers
├── test-runner.js              # 19 automated unit tests for math & state parsers
├── verify-responsive.mjs       # Headless Chrome CDP 14-viewport responsive validation
├── README.md                   # Project manual, installation guide & financial formulas
├── .gitignore                  # Git ignore patterns (.tmp-chrome-profile, .vercel, etc.)
├── api/
│   └── chat.js                 # Vercel serverless function (AI proxy & rate limiter)
└── docs/
    └── superpowers/plans/      # Architecture plans & technical specifications
```

5 The 10 Most Important Files to Understand First

FILE PATH	PRIMARY FUNCTION & DEVELOPER RESPONSIBILITY
`index.html`	The single-page skeleton. Contains all 7 view panels, modal dialogs, and external CDN script tags.
`script.js`	The central brain. Manages the `state` object, persistence, calculations, and UI event listeners.
`style.css`	The presentation engine. Contains the Obsidian Glass design system, CSS variables, and layout breakpoints.
`api/chat.js`	The serverless backend. Handles AI requests, injects financial context when needed, and enforces rate limits.
`vercel.json`	Deployment configuration. Configures clean routing and injects production security headers.
`test-runner.js`	Automated test suite. Executes 19 unit tests to guarantee mathematical precision across all modules.
`verify-responsive.mjs`	Visual quality audit. Uses Chrome DevTools Protocol to inspect 14 viewports for zero overflow.
`README.md`	System documentation. Documents financial formulas, shortcut keys, and local environment setup.
`.gitignore`	Repository hygiene. Prevents testing caches, local credentials, and build artifacts from leaking to Git.
`docs/...plans/...3d.md`	Architectural specification. Detailed technical plan for 3D interactions and performance budgets.

6 External APIs and Services Identified

- **Google Gemini API (Active):** `generativelanguage.googleapis.com` — Primary conversational and analysis model. Configured via `GEMINI_API_KEY`.
- **OpenAI API (Secondary Fallback):** `api.openai.com` — Fallback provider. Configured via `OPENAI_API_KEY`.
- **Lucide CDN (Active):** `unpkg.com/lucide@latest` — Provides modern UI icons without local SVG bloat.
- **Chart.js CDN (Active):** `cdn.jsdelivr.net/npm/chart.js@4.4.1` — Visual canvas rendering for distributions and cashflow.
- **Google Fonts CDN (Active):** `fonts.googleapis.com` — Delivers *Inter* and *JetBrains Mono* typefaces.
- **External Databases (Not Used):** ❌ No external PostgreSQL, Supabase, or MongoDB instance exists. Storage is 100% browser-local.
- **Payment Processors (Not Used):** ❌ No Stripe or Razorpay SDK is connected. Vaultly is an analytical intelligence platform.

7 Proposed Learning Roadmap

MODULE	TOPIC	LEARNING OBJECTIVE & DELIVERABLE
Module 1	**The Blueprint (index.html)**	Single Page Application (SPA) structure, view switching, DOM hierarchy, and modals. *(Current Lesson)*
Module 2	**The Visual Layer (style.css)**	CSS custom properties, glassmorphism filters, responsive grids, and scroll-snapping carousels.
Module 3	**The Brain & Memory (script.js)**	Reactive state management, data modeling, event delegation, and `localStorage` serialization.
Module 4	**Financial Intelligence**	Mathematical formulation of Health Scores, What-If simulation, Anomaly Detection, and Runway.
Module 5	**Interactive 3D & UI**	Spatial canvas particle effects, cursor proximity math, modal lifecycles, and keyboard shortcuts.
Module 6	**The AI Gateway (api/chat.js)**	Serverless API routing, prompt conditioning, streaming/JSON parsing, and rate limiting.
Module 7	**Testing & Verification**	Writing automated unit tests in Node.js, mock testing, and Chrome CDP viewport testing.
Module 8	**Deployment & Security**	Vercel deployment, security headers, CSP rules, performance caching, and production maintenance.

8 Lesson 1: The Entry Point (index.html & The Anatomy of a Single Page Application)

1. What Are We Learning & Why?

We are learning how a **Single Page Application (SPA)** displays 7 completely different screens (Overview, Transactions, Analytics, Budgets, Goals, AI, Settings) *without ever reloading the browser page*. In traditional websites, clicking a link forces the browser to destroy the page and fetch a new HTML file. In modern applications, we download one HTML file once, and use CSS and JavaScript to swap what the user sees instantly.

> **The Beginner Analogy: The Theatre Stage and Scenery Backdrops**
>
> Imagine sitting in a theatre watching a play:
> • In a traditional website, when Scene 2 begins, the entire theatre building is demolished and rebuilt from scratch. It is slow and causes a jarring white flash.
> • In a **Single Page Application**, the stage stays in place. Behind the curtain, all 7 backdrops (the Castle, the Market, the Forest) are already standing. When Scene 2 starts, the stage crew simply shines a spotlight on the Market and turns off the lights on the Castle.
>
> In Vaultly, `index.html` is that single stage containing all 7 panels at once!

2. Real Code Excerpt from `index.html`:

```
<!-- THE NAVIGATION CONTROLS (The Light Switch) -->
<nav class="sidebar-nav">
  <button type="button" class="sidebar-nav-item active" data-view-target="overview">
    <span>Overview</span>
  </button>
  <button type="button" class="sidebar-nav-item" data-view-target="budgets">
    <span>Budgets</span>
  </button>
</nav>

<!-- THE VIEWPORT CONTAINER (The Stage) -->
<main class="dashboard-viewport" id="dashboard">
  <!-- Panel 1: Overview (Currently Spotlighted) -->
  <div class="view-panel active" id="viewOverview" data-view="overview">
    <h1>Your money, in focus.</h1>
  </div>

  <!-- Panel 2: Budgets (Currently in the Dark) -->
  <div class="view-panel" id="viewBudgets" data-view="budgets">
    <h2>Budget Allocations</h2>
  </div>
</main>
```

3. Step-by-Step Breakdown:

- `class="view-panel"`: In `style.css`, this class is defined with `display: none;`. By default, every panel is completely hidden from view.
- `class="view-panel active"`: In CSS, adding the `.active` selector overrides the display: `.view-panel.active { display: block; }`. This is the spotlight!
- `data-view-target="budgets"`: A custom HTML5 data attribute. It acts as an address tag on the button telling JavaScript: "*When clicked, target the panel named 'budgets'*".
- `data-view="budgets"`: The matching address tag on the panel element itself.

4. App Connection Flow (How HTML, CSS, and JS Collaborate):

Step 1 (User Action): The user clicks the navigation button with `data-view-target="budgets"`.

Step 2 (JavaScript Event Listener): In `script.js`, an event listener catches the click and reads `button.dataset.viewTarget`.

Step 3 (DOM Class Manipulation): JS runs `switchView('budgets')`:
• It finds all elements with `.view-panel` and removes the `active` class (turning off all stage lights).
• It finds `.view-panel[data-view="budgets"]` and adds the `active` class (turning on the spotlight).

Step 4 (Instant CSS Rendering): The browser immediately reveals the Budgets screen in 0.0 milliseconds without a page refresh!

5. Practical JavaScript Implementation (from `script.js`):

```
function switchView(viewName) {
  // 1. Turn off spotlights on all panels
  document.querySelectorAll('.view-panel').forEach(panel => {
    panel.classList.remove('active');
  });

  // 2. Turn on spotlight for the matching target panel
  const targetPanel = document.querySelector(`.view-panel[data-view="${viewName}"]`);
  if (targetPanel) {
    targetPanel.classList.add('active');
  }

  // 3. Update the active highlight on the navigation buttons
  document.querySelectorAll('[data-view-target]').forEach(btn => {
    btn.classList.toggle('active', btn.dataset.viewTarget === viewName);
  });
}
```

6. Small Hands-On Task:

Open `index.html` in your editor and locate lines 230–260. Look for the main container `<main class="dashboard-viewport" id="dashboard">` and note how all seven panels (`viewOverview`, `viewTransactions`, `viewAnalytics`, `viewBudgets`, `viewGoals`, `viewAi`, `viewSettings`) are nestled inside.

📝 Chapter 1 Verification Quiz

Answer these 3 questions in your own words to solidify your understanding before Chapter 2:

Question 1: Why does Vaultly load all 7 panels in a single `index.html` file instead of having 7 separate files (like `overview.html`, `budgets.html`, etc.)?

Question 2: What does the CSS class `active` actually do when applied to a `.view-panel` element?

Question 3: If you wanted to add an 8th screen called "Investments", what HTML element and attributes would you need to create in `index.html`?
